Project Title: Stock Market Trend Prediction using AI/ML

Name: Prithvi Koka

Primary Paper:
Author: Himanshu Gupta and Aditya Jaiswal
Title: A Study on Stock Forecasting Using Deep Learning and Statistical Models
Year: 2024
Conference: arXiv
Link: arxiv.org/abs/2502.08728

Introduction: The stock market is a marketplace where buyers and sellers trade ownership stakes in companies. Many various companies are listed on the stock market. However, the stock market is a never ending story of ups and downs.

Problem Statement: There is a way to forecast stocks through looking at historical data. Can lead to a lot of financial gain if we can correctly predict if the stock is going up as well as preventing risk in investing in negative trend stocks.

Motivation and Challenges:
I personally invest in stocks when I can.The stock market can cause us to lose a lot of money especially with high risk investments such as options or shorts. By using algorithms, I can prevent myself from investing in high risk stocks. Can expand this to a bigger market for example stock brokers and advisors.

Key Challenges:
Some key challenges of stock market price prediction are that stock markets are non stationary. They evolve over time due to economic cycles, policy changes, and market evolution. Models training on a bull market have to be retrained to account for a recession. Stock market interactions have a non linear relationship which may make it harder to train. For stocks, models can be prone to overfitting especially due to noise.

Article: Stock Market Prediction Using Machine Learning by Abdulhamit Subasi, Faria Amir, Kholoud Bagedo, Asmaa Shams, Akila Sarirete published in 2021 in Procedia Computer Science

-**Contribution:** This article discusses much of the same things as my main article like model efficiency. It goes into Artificial Neural Networks, K Nearest Neighbor, Support Vector Machine, Decision Trees and their performance on indexes.

-Pros: Takes into account different Machine Learning/AI algorithms on indexes as well as their performance (Measures specific segment of the stock market, for example S&P 500 is an index that tracks the performance of 500 leading companies)

-**Cons:** The reason why this one wouldn't be good to model around is due to the fact it focuses on indexes instead of individual stocks which shows more of just a general trend of the stock market. This would not be good to track trends of individual stocks themselves

Article:
A Comparative Study of Machine Learning Algorithms for Stock Price Predictor Using Insider Training Data by Amitabh Chakravorty and Nelly Elsayed Published in 2025 in University of Cincinnati.

-Contribution:
This article is also similar to my main article but focused on the factor of insider training information for certain companies' stock. They use different algorithms to calculate the efficiency of the models to the actual stock market shares. Focuses on factors to determine stock price such as shares, transaction dates, dollar volume, and type. Dollar volume seems to have the highest relevance.

-Pros: The article talks about using different AI/ML models such as SVM RBF, Random Forest, SVM Polynomial, SVM Linear, K-Means Clustering, and Decision Trees. Discusses the accuracy of these models compared to the actual data and which model is the best for this.

-Cons: The article largely focuses on insider training data. This data is not relevant to us because insider trading is data talking about information that isn't released to the general public. This would be a hard study to replicate for this reason.

Chosen Article: The reason I chose this article to replicate my studies is that it uses some of the ML models that we had studied in class. It delves into LSTM, ARIMA, CNN, and RNN. Explains how each model works and training/testing an S&P 500 index to evaluate the performance of predicting the trend. It even provides graphs to show the actual values compared to the predicted values for each model. What I wanted to differentiate from this article is training an individual stock like Apple instead of an index.

My Methods: Focusing on 4 Algorithms, LSTM, Arima, CNN, and RNN to predict stock market price trends.

$$x'_t = b_0 + b_1 x'_{t-1} + \ldots + b_p x'_{t-p} + \theta_1 e_{t-1} + \ldots + \theta_q e_{t-q} + e_t$$

DIFFERENCED 'x' TIME SERIES (d) PART

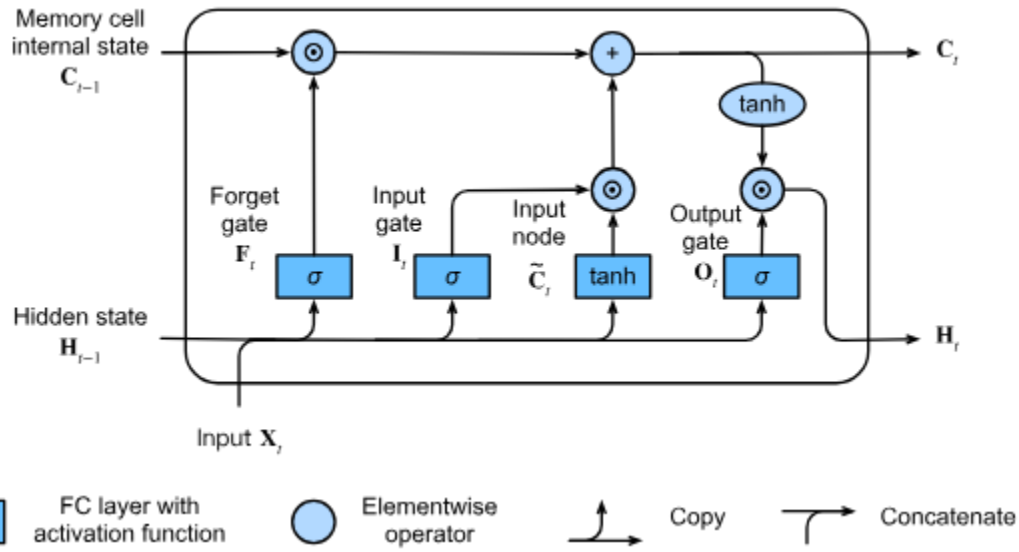
AR (p) PART

MA (q) PART

ARIMA (Autoregressive Integrated Moving Average): A classical statistical model that is focused on forecasting based on time. Focused on three elements, Autoregression, Integration , and Moving Average.

Pros: Interpretable and well understood, works well on small datasets, requiring minimal computing power, good with linear and stationary time series.

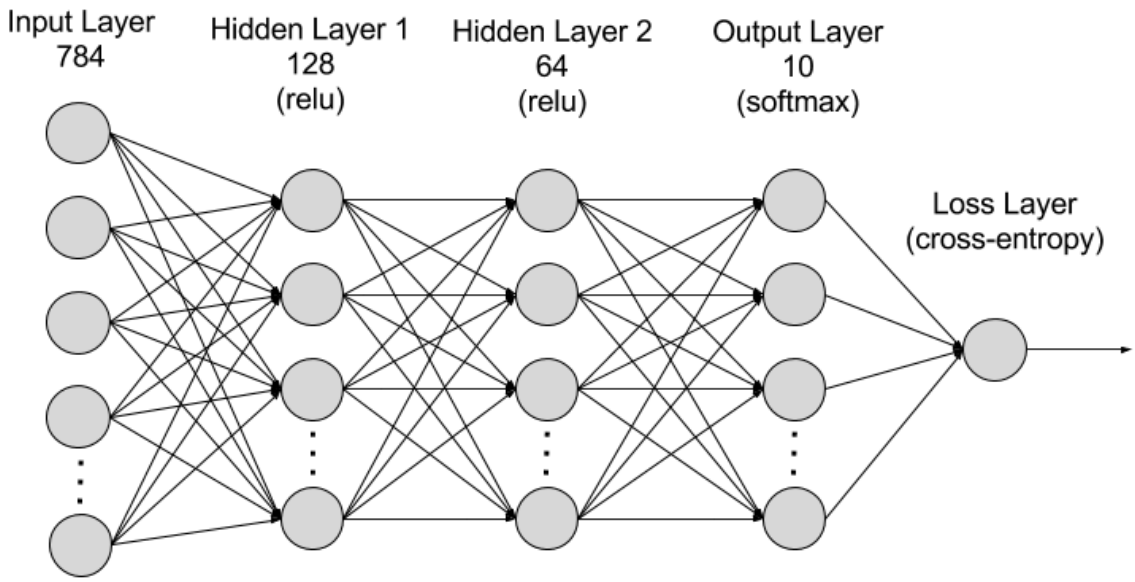
Cons: Assumes a relationship where past values and future values are linear, require manual parameter tuning (p,d,q), struggle with seasonal relationships, cannot capture multiple variable relationships well.



LSTM (Long Short-Term Memory): An advanced RNN variant which solves vanishing gradients by using gating (Input, forget, and output gates). Controls what type of information to retain or discard over a long term period of time.

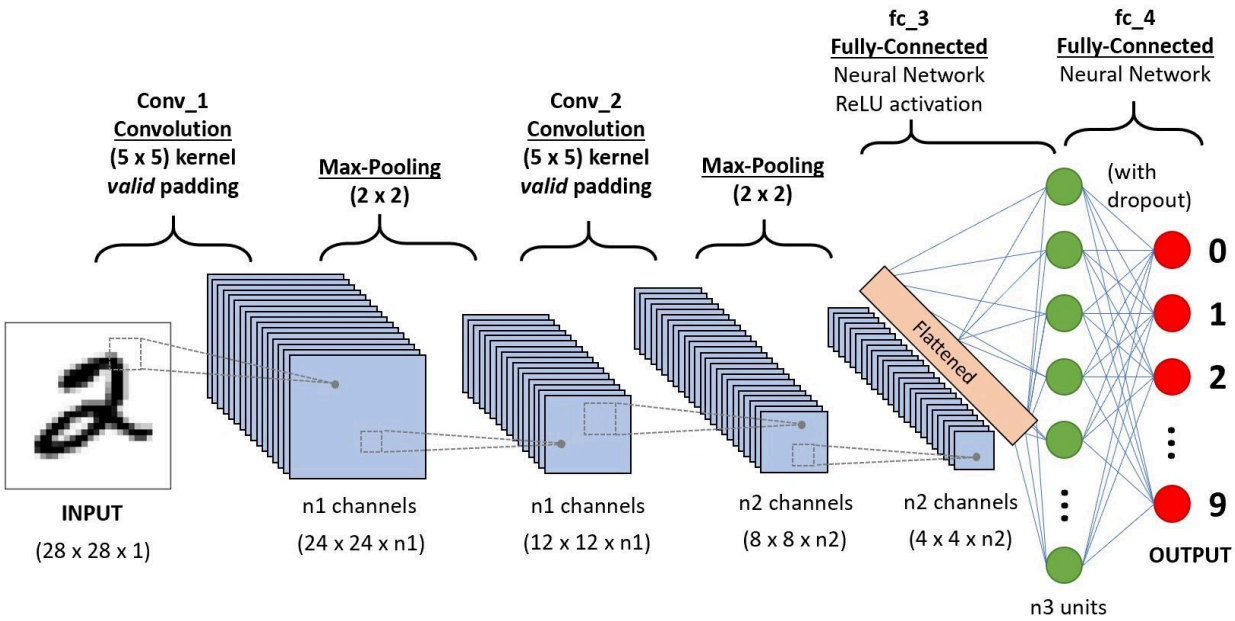
Pros: Does a good job of capturing long term dependency, avoids vanishing gradient, has strong performance on time series.

Cons: Computing is expensive, slow to train, lots of hyperparameters to tune, prone to overfitting, sequential



Recurrent Neural Network: A neural network which is designed for sequential data. Maintains a hidden state which carries information from previous steps and allows it to remember context across a sequence.

> **Pros:** Handles sequential data well, models non linear patterns, works well with variable-length input, effectiveness generalizes.
>
> **Cons:** Suffers from a vanishing/exploding gradient, bad at capturing long term dependencies, slow training.



Convolutional Neural Network: Apply learned patterns across input windows to detect local pattern changes. They apply it to a time series of 1D convolutions and capture short range features.

> **Pros:** Fast training, captures local patterns, less parameters than an RNN, good feature extractor
>
> **Cons:** Limited receptive field, misses long term dependencies, is not sequential, works better with images.

Data: We will be looking at a hot stock that is always in the news. Apple, a tech and electronic company that is a hot commodity in the stock marketplace.

Experiments: We are going to try and replicate the use of ARIMA, LSTM, RNN, and CNN discussed in the article. We will be looking at a dataset from Kaggle for our reference data. This dataset tracks stock data up until February 2020 (Link: https://www.kaggle.com/datasets/jacksoncrow/stock-market-dataset) I couldn't find a dataset with more recent data.

ARIMA:

LSTM:

```
model = Sequential([
    LSTM(128, return_sequences=True, input_shape=(LOOKBACK, X.shape[2])),
    Dropout(0.2),
    LSTM(64, return_sequences=False),
    Dropout(0.2),
    Dense(32, activation='relu'),
    Dense(1)
])

model.compile(optimizer='adam', loss='mse')
model.summary()
```

CNN:

```
model1 = Sequential([
    Conv1D(filters=64, kernel_size=3, activation='relu',
           input_shape=(LOOKBACK, X.shape[2])),
    MaxPooling1D(pool_size=2),
    Dropout(0.2),

    Conv1D(filters=128, kernel_size=3, activation='relu'),
    MaxPooling1D(pool_size=2),
    Dropout(0.2),

    Flatten(),
    Dense(64, activation='relu'),
    Dropout(0.2),
    Dense(1)
])
```

```
model1.compile(optimizer='adam', loss='mse')
model1.summary()
```

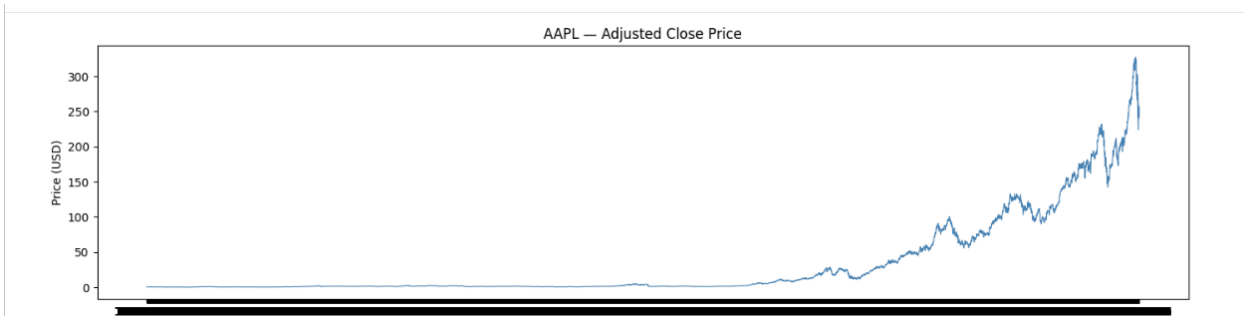
RNN:

```
model2 = Sequential([

    SimpleRNN(128, activation='tanh', return_sequences=True,
              input_shape=(LOOKBACK, X.shape[2])),
    Dropout(0.2),


    SimpleRNN(64, activation='tanh', return_sequences=False),
    Dropout(0.2),


    Dense(32, activation='relu'),
    Dense(1)
])

model2.compile(optimizer='adam', loss='mse')
model2.summary()
```

ARIMA:

```
print('Searching for best ARIMA order')
auto_model = auto_arima(
    close,
    d=1,
    start_p=0, max_p=5,
    start_q=0, max_q=5,
    information_criterion='aic',
    stepwise=True,
    suppress_warnings=True,
    error_action='ignore'
)

best_order = auto_model.order
print(f'\nBest order found: ARIMA{best_order}')
print(auto_model.summary())
```

Conclusion:

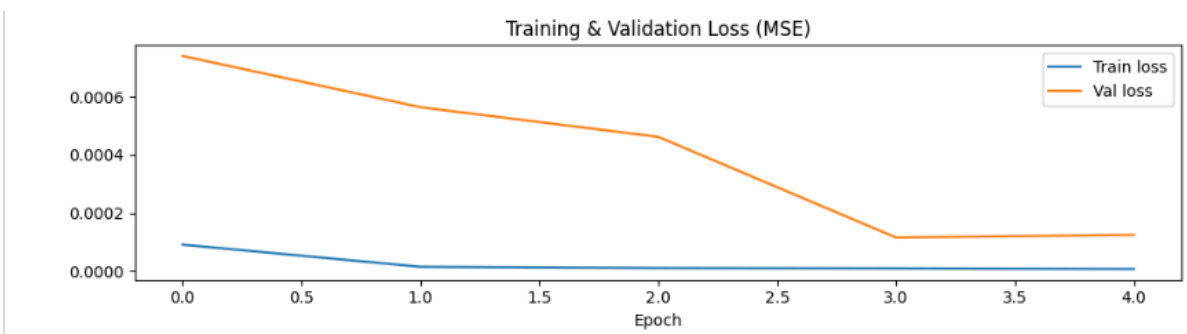


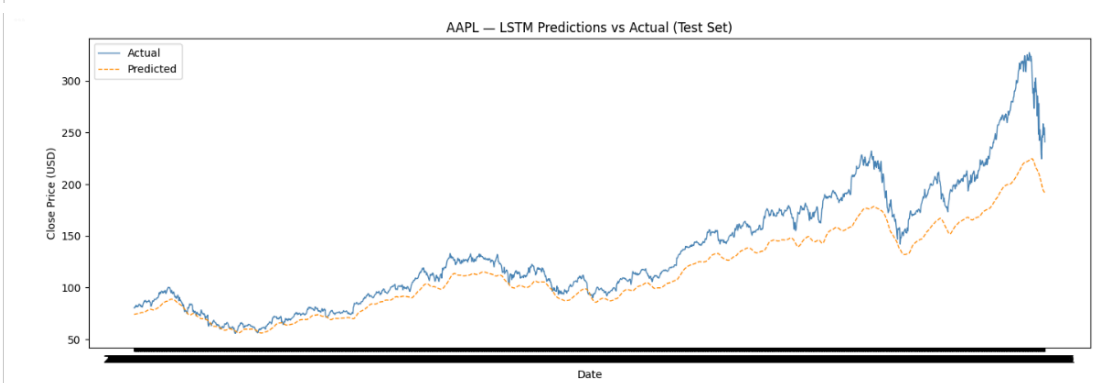
LTSM:

```
  super().__init__(**kwargs)
Model: "sequential_3"
```

Layer (type)	Output Shape	Param #
lstm_2 (LSTM)	(None, 60, 128)	67,072
dropout_7 (Dropout)	(None, 60, 128)	0
lstm_3 (LSTM)	(None, 64)	49,408
dropout_8 (Dropout)	(None, 64)	0
dense_6 (Dense)	(None, 32)	2,080
dense_7 (Dense)	(None, 1)	33

```
Total params: 118,593 (463.25 KB)
Trainable params: 118,593 (463.25 KB)
Non-trainable params: 0 (0.00 B)
```





```
62/62 ━━━━━━━━━━━━━━━━━━━━ 4s 53ms/step
MAE  : $20.05
RMSE : $27.47
MAPE : 12.52%
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 53ms/step
Last close : $240.91
Predicted  : $191.78
```

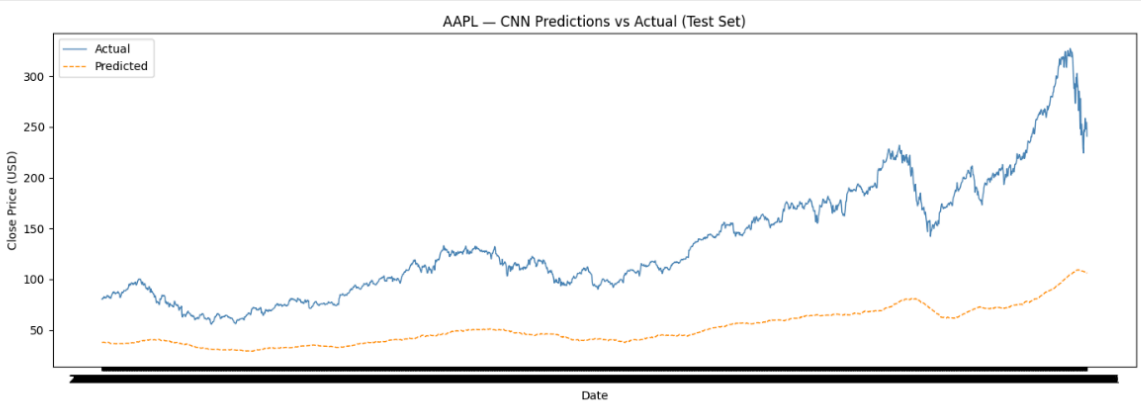
Summary: As we can see, the LSTM model had a somewhat close fitting with the actual values. Its MAE, RSME, and MAPE are not far off. The predicted value went down and by a value that makes some sense in 60 days. (Stock was trending slowly upward and price is still somewhat realistic).

CNN:

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:11
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential_4"
```

Layer (type)	Output Shape	Param #
conv1d_2 (Conv1D)	(None, 58, 64)	448
max_pooling1d_2 (MaxPooling1D)	(None, 29, 64)	0
dropout_9 (Dropout)	(None, 29, 64)	0
conv1d_3 (Conv1D)	(None, 27, 128)	24,704
max_pooling1d_3 (MaxPooling1D)	(None, 13, 128)	0
dropout_10 (Dropout)	(None, 13, 128)	0
flatten_1 (Flatten)	(None, 1664)	0
dense_8 (Dense)	(None, 64)	106,560
dropout_11 (Dropout)	(None, 64)	0
dense_9 (Dense)	(None, 1)	65

```
Total params: 131,777 (514.75 KB)
Trainable params: 131,777 (514.75 KB)
Non-trainable params: 0 (0.00 B)
```




```
62/62 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step
MAE  : $83.13
RMSE : $92.65
MAPE : 60.05%
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 43ms/step
Last close : $240.91
Predicted  : $105.76 ▼
```

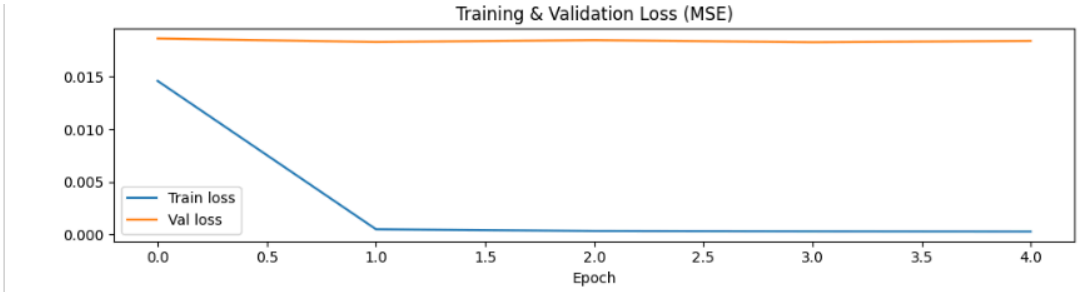
Summary: As we can see CNN didn't perform well, the model's predicted values were much lower than the actual values. The MAE. RSME, and MAPE are far off. The predicted value while trending downward is a much steeper decline than LSTM indicating that the stock incorrectly predicted the price in 60 days.

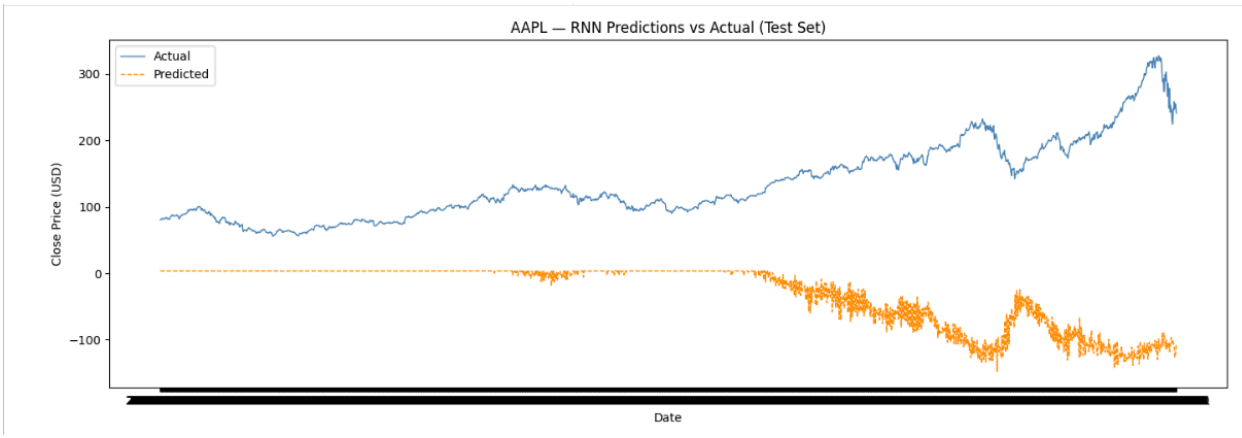
RNN:

```
super().__init__(**kwargs)
Model: "sequential_5"
```

Layer (type)	Output Shape	Param #
simple_rnn_2 (SimpleRNN)	(None, 60, 128)	16,768
dropout_12 (Dropout)	(None, 60, 128)	0
simple_rnn_3 (SimpleRNN)	(None, 64)	12,352
dropout_13 (Dropout)	(None, 64)	0
dense_10 (Dense)	(None, 32)	2,080
dense_11 (Dense)	(None, 1)	33

```
Total params: 31,233 (122.00 KB)
Trainable params: 31,233 (122.00 KB)
Non-trainable params: 0 (0.00 B)
```



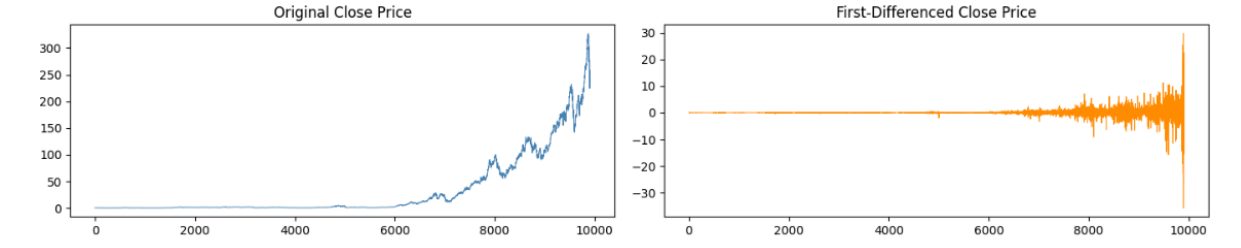


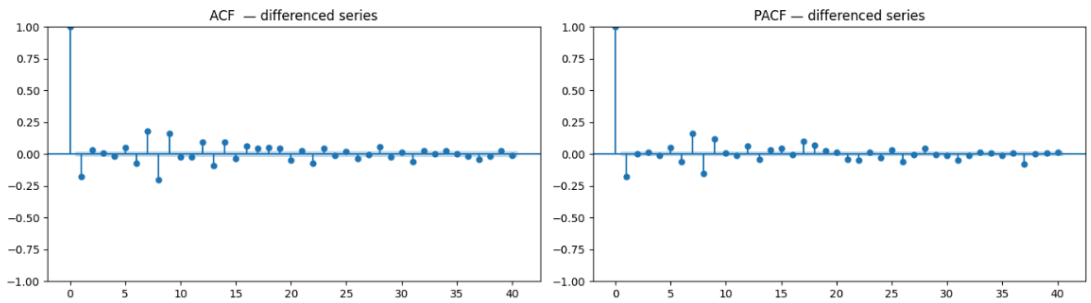

```
62/62 ━━━━━━━━━━━━━━━━━━━━ 2s 20ms/step
MAE  : $163.33
RMSE : $191.63
MAPE : 112.98%
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 243ms/step
Last close : $240.91
Predicted  : $-108.75  ↓
```

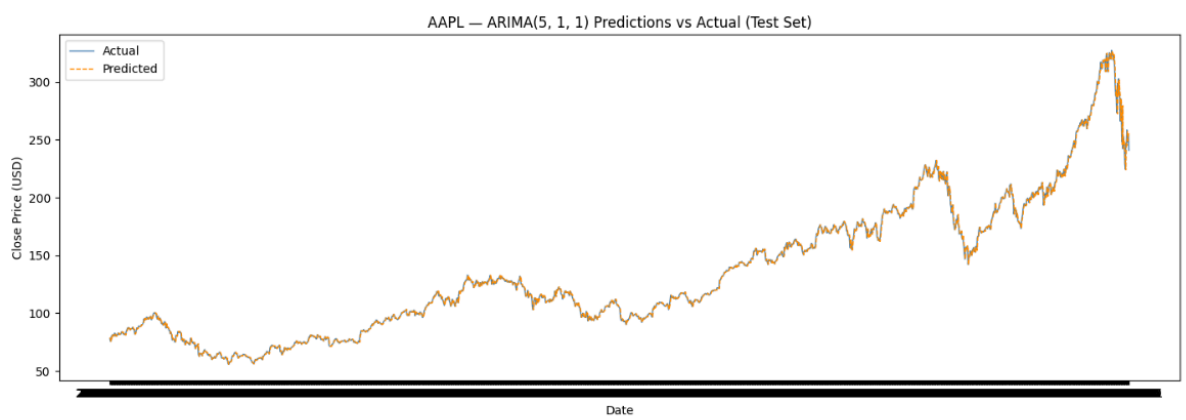
Summary: RNN is even worse than CNN as the predicted values are very low compared to the actual values. The MAE, RMSE, MAPE are all pretty far off from the actual values. The predicted is in the negatives. While it does correctly support the predicted going down, it is very far off from the last close price.

ARIMA:






```
Best order found: ARIMA(5, 1, 1)
                               SARIMAX Results
==============================================================================
Dep. Variable:                      y   No. Observations:                 9909
Model:               SARIMAX(5, 1, 1)   Log Likelihood              -16605.507
Date:                Sun, 26 Apr 2026   AIC                         33227.015
Time:                        21:26:28   BIC                         33284.624
Sample:                             0   HQIC                        33246.524
                               - 9909
Covariance Type:                  opg
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
intercept      0.0477      0.027      1.771      0.077      -0.005       0.100
ar.L1         -0.9580      0.004   -221.422      0.000      -0.966      -0.950
ar.L2         -0.1345      0.003    -47.238      0.000      -0.140      -0.129
ar.L3          0.0192      0.003      5.880      0.000       0.013       0.026
ar.L4          0.0186      0.003      5.553      0.000       0.012       0.025
ar.L5          0.1029      0.002     42.276      0.000       0.098       0.108
ma.L1          0.8094      0.004    185.535      0.000       0.801       0.818
sigma2         1.6720      0.004    452.631      0.000       1.665       1.679
===================================================================================
Ljung-Box (L1) (Q):                   0.59   Jarque-Bera (JB):           4872964.80
Prob(Q):                              0.44   Prob(JB):                         0.00
Heteroskedasticity (H):            2541.04   Skew:                            -1.70
Prob(H) (two-sided):                  0.00   Kurtosis:                       111.59
===================================================================================

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).
```



```
MAE  : $1.65
RMSE : $2.94
MAPE : 1.19%
```

```
Last close : $240.91
Predicted  : $244.48  ↑
```

Summary: As you can see from the close price, it is non stationary. The first-differenced close price is much more stationary as they subtract each new day's price from the previous day's price. This however differentiates later in time. I also charted out the ACF (Autocorrelation Function) which measures how correlated the series is with its own past values at each lag and the PCAF (Partial Autocorrelation Function) which is similar to ACF but removes the influence of intermediates. As you can see from the ACF, there isn't much lag as all values are very close to 0. From the PCAF, you can also see there is a lack of lag as values hover around 0. We then go to the model and as we can see the model is very close to the predicted values. It also isn't far off with the MAE, RSME, and MAPE values. Coincidentally, the predicted value is actually trending upward this time. It is also much closer to the last close price than the other models. This, however, doesn't tell us much and shows that ARIMA values stability over accuracy as the stock actually ended up trending downward.

Overall: It seems that in terms of ranking, I would rate the models from best to worst as: ARIMA-> LSTM-> CNN-> RNN. LSTM incorrectly predicted the stock would go down. CNN predicted the stock would go down, but at a more drastic measure. ARIMA thought the stock would bounce back and actually end up going again which ended up being correct. RNN had the stock value at negative which is wrong to an absurd degree.

As for the reason as to why ARIMA performed so much better, it is probably due to the fact that I had only looked back at the last 60 days for actual training (aka a small time window) instead of looking at the dataset as a whole. I had adjusted it to account for whether the values are stationary (linear) or nonstationary (nonlinear) allowing me much more control. If I had compared a stock that was much more volatile, it would have been much more problematic with the ARIMA model as ARIMA won't be able to correctly account for the changes. Apple is a much more stable stock so it was able to give me a good result.

LSTM was actually the best model in the article I had chosen, but it seems ARIMA performed better. This shows that maybe I could have done different preprocessing or that S&P and other indexes are better for LSTM than individual stocks.

Bibliography (Citations MLA):

"A Study on Stock Forecasting Using Deep Learning and Statistical Models." *arXiv.org*, 2024, arxiv.org/html/2402.06689v1.

Subasi, Abdulhamit, et al. "Stock Market Prediction Using Machine Learning." *Procedia Computer Science*, vol. 194, 2021, pp. 173–179, doi.org/10.1016/j.procs.2021.10.071.

Chakravorty, Amitabh, and Nelly Elsayed. "A Comparative Study of Machine Learning Algorithms for Stock Price Prediction Using Insider Trading Data." *arXiv.org*, 2025, arxiv.org/abs/2502.08728.

Saadeddin, Zaina. "ARIMA for Time Series Forecasting: A Complete Guide." *DataCamp*, 9 Sept. 2024, www.datacamp.com/tutorial/arima.